MDPI

Article

What Standard Evaluation Misses in Small-Model RLVR: Evidence from 30 Countdown Training Runs

Yongsheng Lian [1]

[1] Department of Mechanical Engineering, University of Louisville, Louisville, KY 40292, USA; yongsheng.lian@louisville.edu

Abstract

Reinforcement learning with verifiable rewards (RLVR) is usually evaluated by validation accuracy and standard-prompt benchmarks. We test what these measures miss in 30 Countdown runs of Qwen2.5-1.5B-Instruct: ten seeds each of PPO, GRPO, and DAPO-style training (GRPO with a wider upper clip). Mean Countdown accuracy rose from below 5% before training to 36.9–40.4%, but runs of one method differed by up to 52 points. Eighteen runs shortened their responses early and averaged 26 points below the others. Seven developed a late entropy explosion with degenerate training samples, yet three retained high accuracy under other decoding settings. Early-short and other checkpoints scored 66.5% and 73.3% on standard-prompt GSM8K but 16.1% and 65.3% with the training prompt. Large strict GSM8K gains mainly reflected formatting: flexible accuracy did not rise, and a format instruction alone produced a similar strict gain in the base model. Finally, 22 of 30 checkpoints solved fewer MATH-500 problems within 64 attempts than the base model in any of five sampling runs, including checkpoints with similar single-attempt accuracy. These finite-budget results do not establish loss of underlying capability. Evaluations should report run-to-run variation, inspect training samples, separate formatting from correctness, and test both prompt robustness and repeated-sampling coverage.

Keywords: reinforcement learning with verifiable rewards; large language models; evaluation; training stability; prompt robustness; pass@k; PPO; GRPO

1. Introduction

Reinforcement learning with verifiable rewards (RLVR) can substantially change the reasoning behavior of large language models (LLMs), as illustrated by OpenAI o1 [1] and DeepSeek-R1 [2]. Studies that compare or reproduce RLVR recipes typically judge a trained model by its reward or validation accuracy during training and by benchmark accuracy under a standard prompt afterwards, and many report one training run per configuration, a practice long criticized in deep RL [3,4]. Higher scores on these checks need not imply broadly improved reasoning: gains may depend on output formatting, on the prompt, or on which training run was selected. How much these checks miss has rarely been measured on many independent runs of one recipe.

We measure it on Countdown, a task used in small-model reproductions such as TinyZero [5]. Qwen2.5-1.5B-Instruct was trained in 30 runs: ten seeds each of Proximal Policy Optimization (PPO) [6], Group Relative Policy Optimization (GRPO) [7], and an asymmetric-clipping variant derived from DAPO [8], with the model, data, reward, sampling budget, and update schedule held fixed and the analysis plan set before the first

Received:
Accepted:
Published:

Copyright: © 2026 by the author. Submitted to *AI* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

run. The training prompt requests a worked solution, but the reward checks only the final equation and tag structure, so a model can earn full reward while omitting the explanation. We therefore track response length and entropy during training. Every final checkpoint is evaluated on four benchmarks not used in training (GSM8K, MATH, BBH, and MMLU-Pro), under both a standard prompt and the training prompt. On GSM8K, additional prompts serve two purposes: variants of the training prompt locate which of its components causes a difference between the two prompts, and a format instruction tests whether strict-score gains reflect formatting rather than correctness (Section 4.4). Every answer is scored both strictly, requiring a recognized final-answer format, and flexibly, also accepting a correct answer found by fallback extraction from a completed response. To measure coverage, we draw 64 samples per MATH-500 problem and 32 per GSM8K problem and compute pass@k, sampling the base model with five seeds to estimate sampling variation.

The study makes four findings:

1. **Run-to-run variation exceeded differences between method means.** Early shortening was associated with lower Countdown accuracy, but neither shortening nor high entropy invariably predicted a poor final checkpoint. The prespecified tests did not establish differences between methods.
2. **Standard prompts hid large differences between checkpoints.** Checkpoints from early-short runs scored close to the others on standard-prompt GSM8K but far lower when the same questions were posed in the training prompt. Prompt-component controls located the failure in the prefilled reasoning tag and the literal answer example, a sensitivity already present in the base model.
3. **Strict-score gains on GSM8K were format gains.** Strict accuracy rose by 20–30 points while flexible accuracy did not, and a format instruction alone produced a similar strict gain in the untrained model.
4. **Similar single-sample accuracy could coexist with lower coverage.** With 64 samples per problem, 22 of 30 checkpoints solved fewer MATH-500 problems than the base model in any of five sampling runs. This included 10 of the 12 checkpoints that did not shorten early, a group with near-base median single-attempt accuracy.

Together, these results separate task learning and formatting changes from prompt robustness and broader reasoning gains, and they lead to concrete evaluation practices (Section 6).

2. Related Work

Lian [9] compared PPO, GRPO, and DAPO on Countdown and examined group size, KL regularization, and dynamic sampling. The present study instead asks what standard evaluation misses, using many seeds and controls for prompting, formatting, and coverage.

Run-to-run variation. The outcome of an RL algorithm comparison can depend on the random seed, on implementation details, and on how many runs are compared. Henderson et al. showed that deep RL results vary with the random seed and the environment [3], Engstrom et al. showed that most of PPO's gain in cumulative reward over its predecessor, TRPO, came not from PPO's clipped objective but from implementation details outside the core algorithm, such as reward scaling, learning-rate annealing, and network initialization [10], and Agarwal et al. showed that results reported from few runs carry large statistical uncertainty [4]. RLVR methods also differ in more than their advantage estimator: Hu et al. normalized advantages globally in REINFORCE++ [11], Liu et al. identified a length bias in common GRPO normalization [12], and Yu et al. changed clipping, sam-

pling, loss aggregation, and overlong-response handling in DAPO [8]. We therefore hold these details fixed across methods and report every run.

Response length and entropy. RLVR changes both response length and policy entropy, and neither change is good or bad in itself. Liu et al. showed that the design of the objective can drive response length in opposite directions [13]. Fatemi et al. showed that shorter responses are not necessarily worse: further PPO training of a 1.5B reasoning model more than halved its average response length on four benchmarks while its average accuracy rose slightly, from 57.0% to 58.4% [14]. Entropy is a separate quantity. Most work targets entropy collapse, the loss of exploration as the policy becomes nearly deterministic: Yu et al. designed DAPO's asymmetric clipping to counter it [8], and Cui et al. proposed clipping and KL controls for entropy dynamics [15]. Wu et al. pointed out that the opposite failure, entropy explosion, in which entropy rises until the policy produces near-random text, has received much less attention [16]. We therefore monitor length and entropy separately.

Prompt sensitivity. RLVR-trained LLMs can be highly sensitive to the evaluation prompt. Li et al. showed that evaluating an RL-trained Qwen2.5-7B-Instruct model outside its R1-style training template reduced its MATH-500 accuracy from 73.2% to 3.2% [17]. Zhou et al. found larger losses under rephrased templates in GRPO-trained vision–language models than in their base models, which were not entirely insensitive to those changes [18]. Liu et al. showed that the template already strongly changes the behavior of Qwen2.5 base models [12]. Earlier work on in-context learning found similar effects: Zhao et al. showed that few-shot prompts bias a model toward particular answers [19], and Lyu et al. and Ali et al. showed that models copy labels from demonstrations [20,21].

Formatting versus correctness. Benchmark gains after RLVR can reflect answer format rather than correctness. Shao et al. showed that format and even spurious rewards can elicit gains in some Qwen2.5-Math models [22]. Lyu found that, for four 7B SimpleRL model pairs, 82% of the out-of-domain multiple-choice gain measured under strict answer extraction disappeared when answers outside `\boxed{}` were also read [23]. Gupta and Tripathi found that, in a vision–language model, a single prompt line reproduced much of the RL gain [24]. Yu et al. showed that outcome-only RLVR need not produce causally important written reasoning [25].

Coverage and transfer. Single-attempt accuracy can hide changes in how many problems a model can solve. Brown et al. measured coverage, the fraction of problems solved by at least one of many samples, and showed that it keeps rising over several orders of magnitude of samples [26]. Yue et al. showed that RLVR-trained models can be more accurate in one attempt yet solve fewer distinct problems than their base models given many attempts [27], and Zhou proposed that rarely sampled correct solutions on hard questions are not reinforced and become less likely [28]. Huan et al. found that gains on math need not transfer to other tasks [29]. Broader benefits therefore have to be measured directly.

Our contribution is to measure these effects together, in the same checkpoints and across many independent runs of one recipe, and to relate them to how each run behaved during training.

3. Background: Optimization Methods

The methods share a clipped policy objective but differ in advantage estimation and clipping bounds. For response i and token t, let

$$r_{i,t} = \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})} \tag{1}$$

be the importance sampling ratio between the updated policy π_θ and the sampling policy $\pi_{\theta_{\text{old}}}$, where q is the prompt and $o_{i,t}$ is the t-th token of response i, and let $\widehat{A}_{i,t}$ be its estimated advantage. The clipped contribution is

$$\ell_{i,t} = \min\Big[r_{i,t}\widehat{A}_{i,t},\ \text{clip}(r_{i,t}, 1-\epsilon_{\text{low}}, 1+\epsilon_{\text{high}})\widehat{A}_{i,t}\Big]. \tag{2}$$

All runs average these contributions over generated tokens, excluding padding. PPO and GRPO use symmetric bounds ($\epsilon_{\text{low}} = \epsilon_{\text{high}} = 0.2$); the DAPO-style variant increases only the upper width to 0.28.

PPO trains a critic, a value network initialized from the same pretrained model, alongside the policy. The critic predicts the expected reward at each token, and generalized advantage estimation (GAE) converts these predictions into token-level advantages [6,30]. With the discount and GAE parameters both set to 1, the unnormalized advantage is the response reward minus the critic's prediction at that token; the implementation then normalizes these advantages over valid response tokens. GRPO instead assigns every token in response i the same group-relative advantage [7]:

$$\widehat{A}_i = \frac{R_i - \overline{R}}{s_R + \epsilon_A}, \qquad \overline{R} = \frac{1}{G}\sum_{j=1}^{G} R_j, \tag{3}$$

where G is the number of responses sampled for one prompt, R_i is the response reward, s_R is the sample standard deviation, and ϵ_A prevents division by zero. Equal rewards produce zero advantage; standardization can also turn a small reward difference into a substantial update signal.

Published DAPO combines asymmetric clipping with dynamic sampling, token-level loss aggregation, and overlong-response handling [8]. Our controlled comparison disables dynamic sampling and overlong shaping and uses token-mean aggregation for every method. Thus, "DAPO-style" denotes the asymmetric-clipping variant, not the full published system. Comparing DAPO-style training with GRPO therefore isolates the effect of the upper clip bound, whereas comparing PPO with GRPO changes both the advantage estimator and the use of a critic.

4. Materials and Methods

We fine-tune Qwen2.5-1.5B-Instruct [31] on Countdown and evaluate on four benchmarks not used during fine-tuning: GSM8K [32], MATH [33], BBH [34], and MMLU-Pro [35].

4.1. Training Setup

Countdown requires an equation that uses each supplied number exactly once to reach a target [36]. The reward is additive: 0.9 for a correct equation and 0.1 for the required `<think>` and `<answer>` tag structure. The format check does not assess reasoning content: a correct tagged equation with minimal reasoning can score 1.0, while a formatted but incorrect response scores 0.1. Because GRPO standardizes rewards within each group, this small format credit can carry a large advantage: one format-only response among seven zero-reward responses receives an advantage of about 2.47 (Appendix B). Training uses Verl [37], bfloat16 weights, AdamW, one actor epoch per batch, and rollout temperature 1.0 (Appendix B).

4.2. Controlled Comparison of Training Methods

We run PPO, GRPO, and DAPO-style training with seeds 7–16, giving 30 runs. The seeds control training-question order and response sampling; the same seed gives each

method the same question order, not identical generated responses. All runs share the 50,000-example training set, prompts, reward, and settings in Table 1. Each step generates eight responses to each of 32 prompts. The 256 responses are split into eight mini-batches of 32, giving eight sequential actor updates; after the first, the updated policy differs from the sampling policy, so the clip bounds act. The PPO critic uses learning rate 10^{-5} and the same mini-batch size; its advantage estimator uses no reward discounting or residual decay (Appendix B). The protocol and analysis plan were fixed before the first run.

Table 1. Controlled comparison under a common protocol. Only the final three rows vary by method.

Setting	Value
Reward	0.9 equation correctness + 0.1 tag-format credit
Prompts × responses per step	$32 \times 8 = 256$ trajectories
Actor mini-batch / updates per step	32 responses / 8
Actor LR / warmup / weight decay	10^{-6} / 10 steps / 0.1
KL loss / KL reward / entropy	off / off / 0
Aggregation / response cap	micro-batch token mean / 4,096
Seeds / horizon	paired 7–16 / 300 steps
PPO	GAE, critic, clip 0.2/0.2
GRPO	group-standardized advantage, clip 0.2/0.2
DAPO-style	as GRPO, clip 0.2/0.28, no dynamic sampling

4.3. Training Diagnostics and Checkpoint Selection

We monitor response length to detect departures from the requested worked-solution format. A run is *early short* when its mean length over steps 90–100, rounded to the nearest token, is at most 100 generated tokens. A later increase above this threshold is *response-length recovery*, not necessarily accuracy recovery. With ten runs per method, a difference between two methods can arise from seed-to-seed variation alone. Before the first run, we therefore fixed three questions:

Q1. Does widening the upper clip bound change response shortening? This compares DAPO-style training with GRPO, which differ only in that bound.

Q2. Do critic-based advantages change response shortening? This compares PPO with GRPO, which differ in how advantages are estimated.

Q3. Do the methods end with different Countdown accuracy at step 300?

Shortening is measured by the window mean (the primary outcome) and by the early-short rate. We report two-sided tests and Holm-adjust the two primary window-mean comparisons; a nonsignificant result does not establish equivalence. Analyses added after the runs are labeled as such. Policy entropy is monitored separately; the entropy-explosion criterion in Section 5 was defined after the runs. Every run continues to step 300, and the final checkpoint from every seed is evaluated; neither criterion determines checkpoint selection.

Countdown accuracy measures equation correctness in the last `<answer>` block; format validity measures the required tag structure, independently of correctness. Both use the held-out 1,024-item test split with one sampled response per item (temperature 1.0, top-$p = 0.7$, 4,096-token cap). Unless stated otherwise, reported accuracy is from step 300.

4.4. Benchmark Evaluation

Each checkpoint is evaluated on all 1,319 GSM8K, 5,000 MATH, 6,511 BBH, and 12,032 MMLU-Pro test items. The *standard prompt* uses a benchmark-specific instruction; the *think prompt* places the same question in the Countdown training wrapper, retaining its embedded role labels, reasoning/answer tags, illustrative answer, and assistant prefill. We report

both separately. BBH and MMLU-Pro use custom zero-shot prompts, not their conventional few-shot protocols. Exact prompts appear in Appendix A, and dataset identifiers in Supplementary Note S1.

Greedy evaluation uses vLLM, temperature 0, no repetition penalty, and an 8,192-token output cap, with the base model and all checkpoints on the same hardware. Our *strict* scorer requires a correct answer in a recognized final-answer format. The *flexible* scorer also permits fallback answer extraction from completed responses; length-truncated responses receive no fallback credit. Flexible accuracy is the primary correctness measure, with the extraction limitations stated in Appendix A.

Five GSM8K prompt controls isolate components of the think prompt: remove the answer example, change its value from 42 to 17, remove the opening reasoning tag, remove the complete assistant prefill, or replace the embedded transcript with a clean tag instruction. Questions, model weights, chat wrapping, decoding, and scorers remain fixed, and the controls require no retraining.

Two further controls distinguish formatting from correctness and sampled solution coverage. A prompt-only instruction enforces the final-answer format without changing the base model. Pass@k estimates the probability of at least one correct answer in k sampled responses [26], using 64 samples per MATH-500 item and 32 per GSM8K item. The base model was sampled with five seeds to measure sampling variation; it and all 30 checkpoints share the same sampling environment, temperature 0.6, top-$p = 0.95$, and 4,096-token cap; the estimator and implementation details are in Appendix A.

5. Results

5.1. Run-to-Run Variation in Training Outcomes

Early shortening. All runs began with mean responses of 296.8–438.9 tokens. By steps 90–100, 3 of 10 PPO runs, 7 of 10 GRPO runs, and 8 of 10 DAPO-style runs met the early-short criterion (Table 2); these runs first fell below 100 tokens between steps 11 and 28. Window means fell into two clusters, 26–120 tokens in 19 runs and 1,172–2,283 tokens in 11, with no run in between, so we compared methods with the Mann–Whitney test, which uses ranks rather than means. PPO shortened early in 3 of 10 runs, against 7 of 10 GRPO runs, which suggests that the method affects shortening. Ten seeds per method, however, are too few to establish this: if both methods shortened equally often, a split at least this uneven would still arise by chance in 18% of experiments (Fisher's exact test, $p = 0.18$). The window mean gives a similar result: the Mann–Whitney p-value is 0.035, or 0.071 after Holm correction for testing two pairs of methods. Widening the upper clip showed no sign of an effect: 8 of 10 DAPO-style runs shortened early, against 7 of 10 GRPO runs ($p = 1.0$; window mean $p = 0.19$). In GRPO runs, however, the upper clip was active on only a median 0.06% of response tokens, so this comparison tested a small change. Table 3 lists the prespecified tests.

Table 2. Training outcomes by method (ten runs each). Early short: mean response length over steps 90–100 at most 100 tokens, with exact 95% Clopper–Pearson intervals (%). Window tokens: median (range) of that mean. Entropy explosion: step-300 per-token entropy above 1 nat. Countdown: step-300 validation accuracy, median (range). Every run appears in Table A2.

Method	Early short	Window tokens	Entropy explosion	Countdown (%)
PPO	3/10 (7–65)	2040.4 (34.9–2282.8)	4/10	47.8 (2.1–54.4)
GRPO	7/10 (35–93)	70.9 (26.0–1918.8)	1/10	30.8 (25.8–62.4)
DAPO-style	8/10 (44–97)	59.5 (43.9–1864.1)	2/10	27.3 (25.0–63.6)

Countdown accuracy rose from 0.7–4.8% before training to method means of 39.5% (PPO), 40.4% (GRPO), and 36.9% (DAPO-style) at step 300, whereas runs of the same method ended 36.6–52.3 points apart. The prespecified tests did not detect method differences (Table 3), but the uncertainty is wide: resampling the ten seeds gives 95% intervals of -11.1 to 10.6 points for PPO minus GRPO and -14.3 to 7.8 points for DAPO-style minus GRPO. The 18 early-short runs averaged 28.7%, versus 54.3% for the other 12, a gap of 25.5 points (95% interval 16.5–32.6; $p = 6.0 \times 10^{-5}$; post hoc comparison). This association does not show that shortening caused the accuracy loss. Two early-short runs later recovered in length and reached 55.9% and 63.6% accuracy; the latter also developed the entropy explosion described next. Figure 1 shows the trajectories of all 30 runs.

Entropy explosion. Seven runs became unstable late in training: four PPO runs, one GRPO run, and two DAPO-style runs. Their per-token entropy on the training batch rose above 1 nat between steps 98 and 270 and reached 4.20–8.88 nats at step 300, whereas every other run ended at 0.007–0.753 nats. Their training samples, drawn at temperature 1.0 without truncation, degenerated into unrelated multilingual tokens; in the three group-relative runs, 33–41% of step-300 samples ran to the response cap, sometimes after a tagged answer.

Degenerate training samples coexisted with high evaluation accuracy in the three group-relative explosion checkpoints: they reached 55.5–63.6% on Countdown, 73.2–75.0% on greedy standard-prompt GSM8K, and 52.0–54.5% sampled MATH-500 accuracy, against about 53.9% for the base model. Evaluation differed from training sampling: Countdown validation used top-$p = 0.7$, and benchmarks used greedy decoding or temperature 0.6 with top-$p = 0.95$. These observations suggest decoding-dependent degeneration, but do not isolate the effect of nucleus truncation or locate the excess entropy in a particular part of the token distribution. In PPO, the four explosion runs ended at 2.1–40.9% Countdown accuracy, below every other PPO run (45.3–54.4%); three had also shortened early, so the contributions of the two behaviors cannot be separated. A KL loss toward the initial policy did not consistently prevent explosion in the five selected reruns (Appendix B). With coefficient 0.05 from the first step, the two PPO runs kept long responses and ended without an explosion, but all three group-relative runs still exploded and ended 9.3–22.8 points below their original Countdown accuracy.

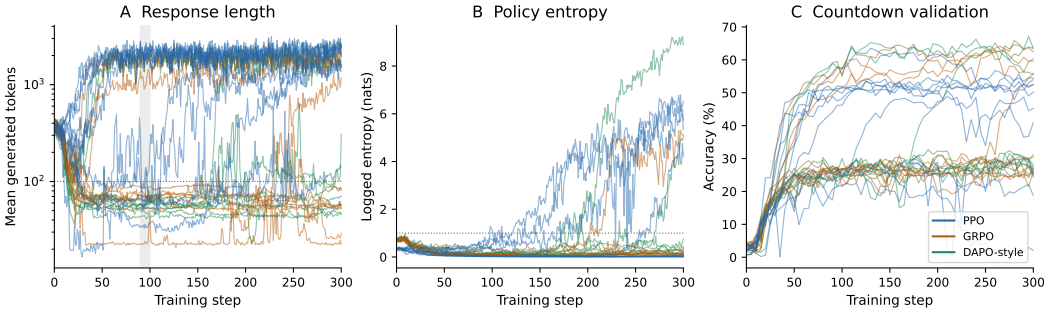


Figure 1. Training trajectories for all 30 runs, with one unsmoothed line per run. (A) Mean rollout length on a logarithmic axis; the shaded window is steps 90–100 and the dotted line is 100 tokens. (B) Logged actor entropy; the dotted line marks the post hoc 1-nat endpoint criterion, not an onset rule. (C) Countdown validation accuracy, sampled every five steps. Entropy aggregation is specified in Appendix B.

Table 3. Prespecified two-sided tests of method differences. Holm correction applies to the two primary window-mean comparisons. None meets the 0.05 threshold after the stated correction; these results do not establish equivalence or exclude practically important differences.

Question	Comparison	Outcome	Test	p
Q1	DAPO-style vs. GRPO	Window mean	Mann–Whitney, Holm	0.19
Q1	DAPO-style vs. GRPO	Early short, 8/10 vs. 7/10	Fisher	1.0
Q2	PPO vs. GRPO	Window mean	Mann–Whitney, Holm	0.071 (unadjusted 0.035)
Q2	PPO vs. GRPO	Early short, 3/10 vs. 7/10	Fisher	0.18
Q3	PPO vs. GRPO	Countdown accuracy	Mann–Whitney	0.58
Q3	DAPO-style vs. GRPO	Countdown accuracy	Mann–Whitney	0.74

What the training method changed. The prespecified tests did not detect differences in final Countdown accuracy or shortening. PPO shortened early less often than GRPO and DAPO-style training and had more entropy explosions (4/10 versus 1/10 and 2/10), but ten runs per method leave substantial uncertainty. Early shortening also took different forms: under the training prompt, early-short PPO checkpoints wrote until the length limit, whereas early-short group-relative checkpoints ended their reasoning almost at once (Section 5.2.1). Because PPO differs from GRPO in both the advantage estimator and the use of a critic, these runs cannot distinguish their contributions.

5.2. Standard Prompts Hid Differences Between Checkpoints

Table 4 reports, for each method, the mean and sample standard deviation of benchmark accuracy over the ten step-300 checkpoints. Under standard prompts the methods look alike: flexible accuracy averages 44.3–47.0%, versus 49.1% for the base model, and the larger PPO spread comes from two degenerate checkpoints (Section 5.2.1). MATH, BBH, and MMLU-Pro show no standard-prompt improvement. We compared each checkpoint with the base model item by item, using the exact McNemar test, which asks whether one model wins the items that only one of the two answers correctly more often than chance (without correction for multiple tests). None of the 12 checkpoints that did not shorten early was significantly more accurate on MATH or BBH, while 6 and 9, respectively, were significantly less accurate, by at most 2.9 and 10.5 points; on MMLU-Pro, one was more accurate (by 1.1 points) and two less. The one gain appeared on BBH under the think prompt, where 7 of the 12 checkpoints that did not shorten early exceeded the base model under the same prompt by 1.6–5.8 points (base 36.5%), and also its standard-prompt score (35.3%); we did not test what drives this gain. Per-seed scores appear in Supplementary Table S1.

Table 4. Benchmark accuracy (%), with equal weight for GSM8K, MATH, BBH, and MMLU-Pro. Parentheses show sample standard deviations across the ten seeds. Strict scoring requires a recognized final-answer format; flexible scoring additionally permits fallback extraction from completed responses.

Method	Early short	Strict standard	Flexible standard	Flexible think
Base model	–	39.1	49.1	41.3
PPO	3/10	43.2 (8.6)	44.3 (8.7)	28.5 (22.3)
GRPO	7/10	42.6 (5.0)	46.7 (2.9)	31.0 (12.9)
DAPO-style	8/10	43.7 (4.4)	47.0 (2.2)	27.6 (11.7)

The think prompt reveals a larger difference between the groups (Table 5). Early-short checkpoints averaged 3.3 points below the others across the four benchmarks under standard prompts, but 22.0 points below them under the think prompt; the GSM8K gap was 49.2 points. These are associations with an outcome observed during training, not evidence that shortening itself caused the errors. Under the wrapper, early-short checkpoints

either ended their reasoning almost at once or wrote until the length limit (Section 5.2.1). They could still answer many of the same questions under standard prompts. Removing the wrapper did not generally improve Countdown performance in the other 12 checkpoints (Supplementary Note S2).

Table 5. Flexible accuracy (%) of early-short and other checkpoints, averaged over checkpoints. Four benchmarks: equal-weight mean of GSM8K, MATH, BBH, and MMLU-Pro. The p-values are from exact two-sided Mann–Whitney tests between the two groups. Without two degenerate PPO checkpoints, early-short checkpoints average 69.0% on standard-prompt GSM8K.

		Four benchmarks		GSM8K	
Checkpoints	n	Standard	Think	Standard	Think
Early short	18	44.7	20.2	66.5	16.1
Other	12	48.0	42.2	73.3	65.3
p		0.13	3.6×10^{-4}	0.004	8.6×10^{-6}

5.2.1. Prompt-Component Controls

We changed individual think-prompt components on GSM8K to identify what triggers the errors. Table 6 groups the 30 final checkpoints by training outcome: early-short group-relative checkpoints, early-short PPO checkpoints, and the remaining checkpoints that did not shorten early. Values average checkpoint accuracies within each group. These outcome-defined groups are not randomized treatment groups.

The think prompt embeds a role-labeled transcript in one user message. It illustrates the answer format with `<answer> 42 </answer>`, a number unrelated to the question, and ends with the assistant prefill `Let me solve this step by step.` followed by an opening `<think>` tag. The five controls (Section 4.4) remove or change these components; the full prompt and every control appear in Appendix A.

Table 6. GSM8K flexible accuracy (%) under the original think prompt and component controls, with standard prompting as the reference condition.

Checkpoint group	Standard prompt	Original wrapper	No example	Example 17	No opening tag	No assistant prefill	Clean tag instruction
Base model ($n = 1$)	72.8	51.6	58.5	51.9	66.5	70.9	68.3
Early-short GRPO/DAPO-style ($n = 15$)	69.0	19.4	27.7	19.5	34.4	35.3	34.4
Early-short PPO ($n = 3$)	53.8	0.1	0.1	0.1	0.0	1.2	15.7
Not early-short ($n = 12$)	73.3	65.3	66.2	64.9	68.5	71.3	68.3

The base model is already sensitive to the wrapper (Table 6): its accuracy drops by 21.2 points. Removing the assistant prefill restores almost all of this loss, whereas removing the answer example restores about a third. Changing the example from 42 to 17 redirects wrong answers toward the new value. Both sensitivities therefore precede RL training.

Training amplified this sensitivity in the early-short checkpoints, in two ways. Early-short GRPO and DAPO-style checkpoints closed the reasoning block almost immediately and lost about 50 points under the wrapper; removing the prefill or the opening tag recovered only a third of this loss. Early-short PPO checkpoints instead wrote until the length limit on almost every item under every wrapper variant. Under standard prompts, most early-short checkpoints kept their accuracy, but two PPO checkpoints were also degraded (Supplementary Note S2).

For a GSM8K item whose correct answer is 14, one early-short GRPO checkpoint generated:

```
Example in prompt: <answer> 42 </answer>
Model response:    </think>
```

```
                    <answer> 42 </answer>

Example in prompt: <answer> 17 </answer>
Model response:     </think>
                    <answer> 17 </answer>
```

Across GSM8K, this checkpoint closed the reasoning tag at once on every item and answered 42 on 58.5% of items, or 17 on 52.6% after the example changed. Removing only the opening tag raised its accuracy from about 6% to 48.1%, so the prefill was a larger trigger than the example. These controls show sensitivity to prompt components, not a causal mechanism.

5.3. Strict-Score Gains Were Format Gains

On GSM8K, the method means for strict accuracy rose from the base model's 33.3% to 53.8–63.3%, whereas flexible accuracy (67.4–70.7%) stayed at or below the base model's 72.8%. Strict scoring credits only answers in a recognized final-answer format; flexible scoring also extracts answers from the response text. Two analyses show that the strict gain reflects formatting, not correctness.

Training's strict-score gain is a format gain. Every strict-correct answer was also flexible-correct, so strict accuracy equals flexible accuracy times the share of correct answers in a recognized format. For the 12 checkpoints that did not shorten early, flexible accuracy was unchanged (73.3% versus 72.8%), but this share rose from 45.7% to 95.7%, which accounts for 36.4 of the 36.9-point strict gain. The change was in how responses ended: the base model ended 55.0% of its responses without a recognized answer marker, usually stating the number in a closing sentence, whereas the checkpoints that did not shorten early did so in at most 15.8%. They mostly ended with `\boxed{}` (65.3% of responses on average), a marker the base model already used in 28.7% of responses, rather than the requested `####` or the `<answer>` tag rewarded in training.

A format instruction alone produces a similar gain. The standard GSM8K prompt asks for the final answer after `####` but does not require it. Appending one sentence that makes this format mandatory (Appendix A) raised the base model's share of responses in a recognized format from 45.6% to 88.9% and its strict accuracy from 33.4% to 58.5%, while flexible accuracy fell from 72.3% to 67.1% (Table 7). A 25-point strict gain thus required neither training nor better answers. The 12 checkpoints that did not shorten early also improved strict accuracy while retaining near-base flexible accuracy; this was not true of all trained checkpoints. Why Countdown training changed how GSM8K responses end was not tested. The result agrees with reports that much of a strict-scored RLVR gain can reflect answer placement [23,24], here on a benchmark outside the training task.

Table 7. GSM8K prompt-only format control on the unchanged base model. Both conditions were evaluated on one two-RTX-4090 workstation.

Condition	Strict accuracy (%)	Flexible accuracy (%)	Recognized format (%)
Standard prompt	33.36	72.25	45.56
Mandatory-format prompt	58.53	67.10	88.93

5.4. Single-Sample Accuracy and Finite-Budget Coverage

With 64 samples per item, five sampling runs of the unchanged base model solved 90.8–92.2% of MATH-500 problems (mean 91.64%). All 30 checkpoints fell below that mean, and 22 fell below the lowest observed base run (Table 8). One PPO checkpoint was 18.0 points below the base mean and reached the 4,096-token cap on 78.5% of samples; the others were 0.04–4.24 points below the mean. Single-attempt accuracy differed between the length groups, but coverage at 64 samples was similar. The 12 checkpoints that did

not shorten early had near-base median pass@1, yet 10 fell below the lowest base pass@64. The five-repeat range is descriptive, not a confidence bound, and the trained checkpoints were sampled only once. Thus, the evidence concerns the probability of finding correct solutions within this sampling budget, not the disappearance of solutions from the model's capabilities. Yue et al. [27] reported that RLVR can improve pass@1 while reducing large-k coverage; our experiment measures the coverage pattern but does not identify its mechanism.

Table 8. MATH-500 sampled accuracy (%) with 64 samples per item under the standard prompt. Base: range over five sampling runs. Checkpoint groups: median, with the range in parentheses. Below base: checkpoints whose pass@64 is below the lowest base run (90.8%). Cap: share of samples that reached the 4,096-token limit. Per-checkpoint values are in Supplementary Table S4.

Model	n	pass@1	pass@64	Below base	Cap
Base (five runs)		53.9–54.0	90.8–92.2		2.3–2.4
Not early-short	12	53.9 (51.6–54.6)	89.7 (88.8–91.6)	10	3.0–7.6
Early-short	18	50.1 (8.0–54.5)	90.0 (73.6–91.0)	12	1.2–78.5

GSM8K is less informative because coverage was near its ceiling: the base model solved 95.9% of items within 32 samples, 24 of the 30 checkpoints were slightly above it (by at most 1.3 points), and the largest losses were 7.9 points for one PPO checkpoint and 3.1 points for one DAPO-style checkpoint. Sampled strict GSM8K pass@1 rose by a median of 23.2 points over the base model's 37.1%, again reflecting the format change of Section 5.3 rather than better answers.

6. Discussion

Each usual check missed something that another measurement exposed: standard prompts made checkpoints of very different robustness look alike, strict scoring credited formatting as correctness, single-sample accuracy hid lost coverage, and training entropy did not identify worse checkpoints. The study is therefore an evaluation diagnosis, not an algorithm ranking.

Variation within each method was much larger than the differences between method means. The prespecified tests did not establish method differences, but their uncertainty leaves practically important effects unresolved. A PPO difference could not be attributed to the critic alone, and the DAPO-style contrast tested a rarely active upper clip. High entropy and degenerate training rollouts did not invariably imply poor accuracy under other decoding settings. The selected KL reruns gave mixed outcomes rather than a general remedy.

RLVR-trained and Qwen2.5 base models are known to be prompt-sensitive [12,17,18]. Here, the direction of the effect depended on the run: under the training prompt, the checkpoints that did not shorten early exceeded the base model on average on GSM8K, whereas early-short checkpoints lost about 50 points relative to their own standard-prompt accuracy. Unlike the template-mismatch result of Li et al. [17], this failure occurred in the training template itself. Direct numerical comparisons across these studies would confound models, tasks, and prompt changes. The component controls identified sensitivity to the reasoning prefill and the answer example; copying the latter resembles previously reported demonstration-label copying [20,21]. Whether unprinted reasoning occurred remains open [25]. Because a format instruction alone reproduced much of the strict GSM8K gain, format adherence should be reported separately from correctness.

The checkpoints that did not shorten early learned Countdown while staying within 2.2 points of the base model on the four-benchmark average, but retention was incomplete:

BBH fell by a median of 2.9 points, early-short checkpoints lost 6.3 points on GSM8K and MATH, and most checkpoints lost MATH-500 coverage at 64 samples. Narrow RLVR thus added a task skill at a small cost elsewhere, which single-sample evaluation partly hides.

Limitations. The evidence covers one 1.5B model and one arithmetic training domain. Ten seeds per method detect only large differences, and item-level tests do not replace independent runs. The DAPO-style variant omits parts of published DAPO. The length and entropy criteria are operational, and the latter was defined after the runs. The KL tests used one run per seed and coefficient, and reruns of a seed did not reproduce exactly. The think prompt is a robustness probe whose component controls cover only GSM8K; BBH and MMLU-Pro use custom zero-shot prompts; and the error rate of the flexible GSM8K fallback was not estimated.

6.1. Practical Guidance for Training Small Models

1. **Pilot several seeds.** Confirm that each intended mechanism is active, for example a nonzero clip fraction, and fix continuation and checkpoint-selection rules in advance.
2. **Monitor beyond accuracy.** Log correctness, length, entropy, and clip fractions, read training samples, and report decoding settings with every accuracy, because degeneration may appear only under untruncated sampling.
3. **Inspect the normalized reward.** Check whether format credit, after advantage normalization, rewards incorrect outputs.
4. **Evaluate robustly.** Test each checkpoint under both standard and training prompts, report strict and flexible scores separately, and add repeated-sampling coverage for claims of broader reasoning.

Deployment may justify selecting a robust seed; a scientific comparison should still report every run and the selection rule.

7. Conclusion

Across 30 Countdown RLVR runs of a 1.5B model, in-domain accuracy improved, but the benefits were not uniform across runs, prompts, or evaluation measures. Run-to-run variation exceeded differences between method means. Early shortening was associated with lower accuracy, while high entropy and degenerate training samples could coexist with good performance under other decoding settings. Think-prompt controls identified sensitivity to the reasoning prefill and literal answer example. Strict GSM8K gains mainly reflected formatting, and 22 checkpoints had lower MATH-500 coverage at 64 samples than all five base-model sampling runs, including checkpoints with near-base single-attempt accuracy. These observations concern measured performance, not proof of lost underlying capability. Reliable assessment therefore requires multiple training runs, inspection of training samples, separate correctness and formatting measures, repeated-sampling coverage, and evaluation under both standard and training prompts.

Supplementary Materials: The following supporting information can be downloaded at: https://www.mdpi.com/article/10.3390/ai1010000/s1, Table S1: Flexible accuracy for the base model and all 30 checkpoints; Table S2: Strict accuracy for the base model and all 30 checkpoints; Table S3: GSM8K flexible accuracy under the prompt-component controls; Table S4: Sampled pass@k for the base model and all 30 checkpoints; Notes S1–S4: evaluation environment and data identifiers, additional prompt diagnostics, additional training diagnostics, and additional coverage details.

Author Contributions: Y.L. performed all aspects of the research and manuscript preparation and approved the final manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The reproducibility artifact contains training and evaluation code, configurations, compact results, dataset identifiers, environment records, and checkpoint hashes. The public GitHub repository currently hosts an earlier artifact version; the 30-run results and the accompanying supplementary material will be released there before submission. A DOI-bearing archive is planned. Large model checkpoints, raw training logs, and per-item generation archives are available from the corresponding author on reasonable request where redistribution terms permit. Third-party datasets and the Verl framework are available through their respective citations.

Acknowledgments: Model evaluation used the LARCC cluster at the University of Louisville. Computational infrastructure was supported in part by the U.S. National Science Foundation (NSF) under grants OAC2430270 and OAC2322248 and the University of Louisville's Research Computing team. During preparation of this manuscript, the author used OpenAI Codex and Anthropic Claude (Claude Code) for language editing, manuscript restructuring, code-assisted consistency checks, and preparation of evaluation and analysis scripts. The author reviewed and edited all output and takes full responsibility for the content of this publication.

Conflicts of Interest: The author declares no conflicts of interest.

Appendix A. Evaluation Prompts and Scoring

Decoding. Each prompt below was passed as the content of one user message through the model's chat template. Greedy evaluation used temperature 0 and an 8,192-token limit; pass@k used temperature 0.6, top-p $= 0.95$, and a 4,096-token cap. Hardware, software versions, checkpoint hashes, and dataset revisions are listed in Supplementary Note S1.

Exact prompt strings. Braced fields denote item-specific substitutions. The standard GSM8K user content was:

```
{question} Let's think step by step and output the final answer after "####".
```

The prompt-only format control appended the following text to that standard user content:

```
Your response must end with exactly one separate final line of the form
"#### N", where N is replaced by only the final numeric answer. This final
line is mandatory and must be the last line of your response.
```

The Countdown standard-prompt control used the following user content, with the same task sentence as the training wrapper:

```
Using the numbers {nums}, create an equation that equals {target}. You can
use basic arithmetic operations (+, -, *, /) and each number can only be
used once. Show your work, then give the final equation on the last line in
the form "Answer: <equation>", for example "Answer: (1 + 2) / 3".
```

Both Countdown prompts were scored with one rule: the candidate equation is whichever occurs last of the final `<answer>` block, the final `\boxed{}`, and the final `Answer:` marker, and it is checked with the training reward's equation validator. Under the wrapper, this rule agreed exactly with the training reward's correctness for all 30 checkpoints. The standard MATH user content was:

```
{problem}
Please reason step by step, and put your final answer within \boxed{}.
```

The standard BBH user content was:

```
{input}

Let's think step by step, and end your response with
"So the answer is <answer>."
```

The standard MMLU-Pro user content was:

```
The following is a multiple choice question about {category}. Think step by
step and then finish your answer with "the answer is (X)" where X is the
correct letter choice.

Question: {question}
Options:
{lettered options}
Answer: Let's think step by step.
```

The common think-condition string, including the wording and quotation marks inherited from the training wrapper, was passed as a *single user-message content string*:

```
system: You are a helpful assistant. You first thinks about the reasoning
process in the mind and then provides the user with the answer."
User: {question} Show your work in <think> </think> tags. And return the
final answer in <answer> </answer> tags, for example
<answer> {example} </answer>."
Assistant: Let me solve this step by step.
<think>
```

The illustrative answer is 42 for GSM8K, $\frac{3}{4}$ for MATH, C for MMLU-Pro, and the target of a different item from the same task for BBH. The MMLU-Pro question includes the lettered options and an instruction to answer with the option letter.

Prompt-component controls. The *no-example* condition deleted only the phrase beginning `for example`; the *example-17* condition replaced the literal example answer 42 with 17. The *no-opening-tag* condition deleted only the final prefilled `<think>` line while retaining `Assistant: Let me solve this step by step.`; the *no-assistant-prefill* condition deleted both of those final lines. The *clean-tag-instruction* condition removed the embedded system/user/assistant transcript and used the following single user-message content:

```
{question}
Show your work between <think> and </think> tags, then put only the final
numeric answer between <answer> and </answer> tags.
```

All controls used the same decoding as the primary prompts, for the base model and all 30 checkpoints.

Scoring. For GSM8K, strict scoring takes the last explicit marker among `####`, `\boxed{}`, and `<answer>`; flexible scoring accepts a correct strict answer and otherwise takes the last number in a response that did not end by length truncation. MATH applies a symbolic verifier to the last `\boxed{}` or `<answer>` content (strict) or to expressions parsed from a completed response (flexible). BBH and MMLU-Pro take the last recognized final-answer marker (strict) or otherwise the last parenthesized option letter of a completed response (flexible). The GSM8K fallback can credit an incidental final number, and it credits a correct last number even when an explicit marker holds a different one. Rescoring with a fixed rule that ignores the gold answer (the explicit marker when one is present, otherwise the last number) changed mean flexible accuracy of the base model, the early-short checkpoints, and the other checkpoints by at most 1.1 points under either prompt; we did not hand-audit fallback-credited answers.

https://doi.org/10.3390/ai1010000

Sampled solution coverage. For an item with c correct responses among n samples, pass@k is estimated as

$$\widehat{\text{pass@}k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}, \qquad 1 \le k \le n, \tag{A1}$$

with the numerator taken as zero when $n - c < k$. Benchmark scores average this estimate over items. We use $n = 64$ for MATH-500 and $n = 32$ for GSM8K, and the base model and all checkpoints were sampled on the same machine. The MATH-500 coverage loss is a net change: apart from one PPO checkpoint, each checkpoint never solved 12–28 items that the base model solved and solved 4–11 that it never solved, whereas resampling the base model lost 7–12 and gained 3–8. Truncation does not explain the loss, because early-short checkpoints reached the 4,096-token cap less often than checkpoints that did not shorten early yet lost similar coverage (Supplementary Note S4).

Appendix B. Training Details

Training configuration. All three methods use group size 8, 32 prompts per step (256 responses), actor mini-batches of 32 responses (eight sequential actor updates per step, one epoch), a token-mean loss within each micro-batch with micro-batch losses weighted by size, actor learning rate 10^{-6} with 10 warmup steps, gradient clipping 1.0, no KL loss or KL reward penalty, entropy coefficient 0, no dynamic sampling, a 4,096-token response cap, rollout temperature 1.0 without top-p truncation, and 300 steps. The reward is 0.9 correctness plus 0.1 format credit. GRPO and PPO clip the importance ratio at 1 ± 0.2; DAPO-style training uses lower and upper bounds of 0.2 and 0.28. Because updates 2–8 of each step are made with a policy that has already moved, the ratio departs from 1 and the clip bounds act. Seeds 7–16 pair the data and rollout seeds within each run. Validation uses the 1,024-item Countdown test split every five steps. The protocol, including the analysis plan, was fixed before the first run. One run (GRPO seed 7) stopped after step 290 when a disk filled and was resumed from its step-250 checkpoint.

PPO advantage estimation. For token state s_t, immediate reward R_t, and learned value function V_ϕ, generalized advantage estimation uses

$$\delta_t = R_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \qquad \widehat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}. \tag{A2}$$

The discount factor γ and residual-decay parameter λ are both 1, so residuals are summed through the end of the response. The critic is initialized from the same model, trained with learning rate 10^{-5} (10 warmup steps, weight decay 0.1), and uses the same mini-batch size as the actor, giving eight critic updates per step.

Criteria and clip diagnostics. No run has a window mean between 96.0 and 119.6 tokens, so the early-short classification does not depend on the 100-token boundary. Entropy is the per-token Shannon entropy (nats) of the actor on each training batch, averaged over response tokens; the step-300 values separate into 0.007–0.753 and 4.20–8.88 nats, so any threshold between them gives the same classification. The upper clip fraction is the share of response tokens with positive advantage and ratio above $1 + \epsilon_{\text{high}}$; the lower clip fraction is the share with negative advantage and ratio below $1 - \epsilon_{\text{low}}$.

Format-reward normalization. For rewards $(0, 0, 0, 0, 0, 0, 0, 0.1)$, the mean is 0.0125 and the implementation's Bessel-corrected sample standard deviation is approximately 0.0354. Equation 3 therefore assigns the format-only response advantage $(0.1 - 0.0125)/(0.0354 + \epsilon_A) \approx 2.47$ for negligible ϵ_A.

https://doi.org/10.3390/ai1010000

KL loss and the entropy explosion. These runs were added after the main experiment. The loss adds β times a low-variance estimate of the KL divergence from the initial policy to the actor loss; every other setting is the original run's. Five explosion seeds were retrained from step 0 with the KL loss on (Table A1). With $\beta = 0.05$, the two PPO seeds, whose originals had shortened early and then exploded, kept long responses and ended without an explosion. The three group-relative seeds still exploded, with onset at steps 116–276, and ended 9.3–22.8 points below their original Countdown accuracy. With $\beta = 0.001$, entropy exploded in two of five runs, and two group-relative runs shortened early although their originals had not. A penalty this small changes the loss little, so the different outcomes of these runs mainly show that a seed does not fix whether a run fails. Each configuration has a single run, so these results bound what a KL loss can be expected to do rather than estimate its effect. Runs resumed from a checkpoint before entropy onset gave the same picture: $\beta = 0.05$ delayed or softened the explosion in most runs but did not prevent it (Supplementary Note S3).

Table A1. Entropy-explosion seeds retrained from step 0 with a KL loss of coefficient β. No KL: the original run (Table A2). Window tokens, step-300 H, and Countdown are defined as in Table A2; H above 1 nat is an entropy explosion. One run per configuration.

Method and seed	KL	Window tokens	Step-300 H	Countdown (%)
PPO 8	none	96.0	5.34	2.1
	0.001	93.4	6.09	9.9
	0.05	1214.5	0.30	41.4
PPO 16	none	34.9	5.72	18.8
	0.001	1924.0	0.02	47.4
	0.05	1161.6	0.22	43.6
GRPO 13	none	1172.0	5.09	59.6
	0.001	67.5	0.31	30.0
	0.05	898.1	4.12	41.9
DAPO-style 11	none	1864.1	8.88	55.5
	0.001	53.2	0.59	30.0
	0.05	549.1	7.09	46.2
DAPO-style 15	none	59.0	5.07	63.6
	0.001	976.6	5.03	63.8
	0.05	560.3	7.96	40.8

Appendix C. Complete Training Results

Table A2 reports every run; no run was stopped early, dropped, or replaced.

Table A2. All 30 training runs. Window tokens is the mean response length over steps 90–100. Step-300 H is the per-token categorical entropy (nats) of the training batch at step 300; an entropy explosion is step-300 H above 1 nat. Countdown is validation accuracy at step 300. Clip fractions are percentages of response tokens, averaged over the 300 training steps.

Method and seed	Window tokens	Early short	Step-300 H	Entropy explosion	Countdown (%)	Upper clip (%)	Lower clip (%)
PPO 7	2060.0	No	0.01	No	52.4	0.07	0.11
PPO 8	96.0	Yes	5.34	Yes	2.1	0.76	2.21
PPO 9	2220.8	No	0.02	No	45.3	0.08	0.12
PPO 10	66.1	Yes	6.52	Yes	24.3	0.51	1.59
PPO 11	2174.4	No	0.14	No	54.0	0.08	0.10
PPO 12	2020.7	No	0.02	No	52.6	0.07	0.11
PPO 13	2111.2	No	0.01	No	50.2	0.06	0.11
PPO 14	119.6	No	4.20	Yes	40.9	0.21	0.43
PPO 15	2282.8	No	0.01	No	54.4	0.07	0.09
PPO 16	34.9	Yes	5.72	Yes	18.8	0.92	2.17
GRPO 7	1811.0	No	0.10	No	60.3	0.03	0.10
GRPO 8	69.5	Yes	0.39	No	26.1	0.07	0.70
GRPO 9	87.3	Yes	0.06	No	25.8	0.04	0.38
GRPO 10	52.8	Yes	0.11	No	31.1	0.08	0.54
GRPO 11	1918.8	No	0.05	No	62.4	0.03	0.11
GRPO 12	71.6	Yes	0.10	No	25.8	0.05	0.44
GRPO 13	1172.0	No	5.09	Yes	59.6	0.13	0.38
GRPO 14	65.5	Yes	0.18	No	55.9	0.06	0.37
GRPO 15	70.3	Yes	0.09	No	26.9	0.04	0.40
GRPO 16	26.0	Yes	0.06	No	30.5	0.12	0.61
DAPO-style 7	1843.2	No	0.36	No	63.5	0.01	0.12
DAPO-style 8	43.9	Yes	0.24	No	27.0	0.07	0.63
DAPO-style 9	48.7	Yes	0.42	No	28.1	0.06	0.58
DAPO-style 10	61.3	Yes	0.75	No	27.6	0.04	0.48
DAPO-style 11	1864.1	No	8.88	Yes	55.5	0.02	1.68
DAPO-style 12	65.5	Yes	0.17	No	25.0	0.04	0.40
DAPO-style 13	54.6	Yes	0.07	No	26.5	0.06	0.56
DAPO-style 14	60.1	Yes	0.11	No	26.9	0.04	0.44
DAPO-style 15	59.0	Yes	5.07	Yes	63.6	0.02	0.47
DAPO-style 16	57.3	Yes	0.24	No	25.1	0.05	0.57

References

1. OpenAI. Learning to Reason with LLMs. Available online: https://openai.com/index/learning-to-reason-with-llms/ (accessed on 12 October 2026).
2. DeepSeek-AI; Guo, D.; Yang, D.; Zhang, H.; Song, J.; Wang, P.; Zhu, Q.; Xu, R.; Zhang, R.; Ma, S.; Bi, X.; et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *Nature* **2025**, *645*, 633–638. https://doi.org/10.1038/s41586-025-09422-z.
3. Henderson, P.; Islam, R.; Bachman, P.; Pineau, J.; Precup, D.; Meger, D. Deep Reinforcement Learning That Matters. *Proc. AAAI Conf. Artif. Intell.* **2018**, *32*, 3207–3214. https://doi.org/10.1609/aaai.v32i1.11694.
4. Agarwal, R.; Schwarzer, M.; Castro, P.S.; Courville, A.C.; Bellemare, M.G. Deep Reinforcement Learning at the Edge of the Statistical Precipice. *Adv. Neural Inf. Process. Syst.* **2021**, *34*, 29304–29320.
5. Pan, J.; Zhang, J.; Wang, X.; Yuan, L.; Peng, H.; Suhr, A. TinyZero: Minimal Reproduction of DeepSeek R1-Zero. Available online: https://github.com/Jiayi-Pan/TinyZero (accessed on 12 October 2026).
6. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal Policy Optimization Algorithms. *arXiv* **2017**, arXiv:1707.06347.
7. Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y.K.; Wu, Y.; et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv* **2024**, arXiv:2402.03300.
8. Yu, Q.; Zhang, Z.; Zhu, R.; Yuan, Y.; Zuo, X.; Yue, Y.; Dai, W.; Fan, T.; Liu, G.; Liu, L.; et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *Adv. Neural Inf. Process. Syst.* **2025**, *38*; arXiv:2503.14476.
9. Lian, Y. Comparative Analysis and Parametric Tuning of PPO, GRPO, and DAPO for LLM Reasoning Enhancement. *arXiv* **2025**, arXiv:2512.07611.

10. Engstrom, L.; Ilyas, A.; Santurkar, S.; Tsipras, D.; Janoos, F.; Rudolph, L.; Madry, A. Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO. In *Proceedings of the International Conference on Learning Representations*; 2020.
11. Hu, J.; Liu, J.K.; Xu, H.; Shen, W. REINFORCE++: Stabilizing Critic-Free Policy Optimization with Global Advantage Normalization. *arXiv* **2025**, arXiv:2501.03262.
12. Liu, Z.; Chen, C.; Li, W.; Qi, P.; Pang, T.; Du, C.; Lee, W.S.; Lin, M. Understanding R1-Zero-Like Training: A Critical Perspective. In *Proceedings of the Conference on Language Modeling (COLM)*; 2025; arXiv:2503.20783.
13. Liu, F.; Yin, Y.; Shi, P.; Yang, S.; Zeng, Z.; Qiu, H. Length-Unbiased Sequence Policy Optimization: Revealing and Controlling Response Length Variation in RLVR. *arXiv* **2026**, arXiv:2602.05261.
14. Fatemi, M.; Rafiee, B.; Tang, M.; Talamadupula, K. Concise Reasoning via Reinforcement Learning. *arXiv* **2025**, arXiv:2504.05185.
15. Cui, G.; Zhang, Y.; Chen, J.; Yuan, L.; Wang, Z.; Zuo, Y.; Li, H.; Fan, Y.; Chen, H.; Chen, W.; et al. The Entropy Mechanism of Reinforcement Learning for Reasoning Language Models. *arXiv* **2025**, arXiv:2505.22617.
16. Wu, J.; Huang, K.; Wu, J.; Zhang, A.; Wang, X.; He, X. Quantile Advantage Estimation: Stabilizing RLVR for LLM Reasoning. In *Proceedings of the International Conference on Learning Representations*; 2026; arXiv:2509.22611.
17. Li, Y.; Pan, Z.; Lin, H.; Sun, M.; He, C.; Wu, L. Can One Domain Help Others? A Data-Centric Study on Multi-Domain Reasoning via Reinforcement Learning. *arXiv* **2025**, arXiv:2507.17512.
18. Zhou, P.; Tang, Z.; Peng, X.; Zhou, C.; Moukheiber, L.; Ma, Y.; Xu, B.; Song, J.; Wan, Z.; Zhao, W.; et al. Improving Generalization Robustness of Multimodal RLVR. *arXiv* **2026**, arXiv:2608.08802.
19. Zhao, Z.; Wallace, E.; Feng, S.; Klein, D.; Singh, S. Calibrate Before Use: Improving Few-Shot Performance of Language Models. In *Proceedings of the 38th International Conference on Machine Learning*; PMLR: 2021; Volume 139, pp. 12697–12706.
20. Lyu, X.; Min, S.; Beltagy, I.; Zettlemoyer, L.; Hajishirzi, H. Z-ICL: Zero-Shot In-Context Learning with Pseudo-Demonstrations. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*; Association for Computational Linguistics: Toronto, Canada, 2023; pp. 2304–2317. https://doi.org/10.18653/v1/2023.acl-long.129.
21. Ali, A.A.; Wolf, L.; Titov, I. Mitigating Copy Bias in In-Context Learning through Neuron Pruning. In *Findings of the Association for Computational Linguistics: EACL 2026*; Association for Computational Linguistics: Rabat, Morocco, 2026; pp. 230–251. https://doi.org/10.18653/v1/2026.findings-eacl.13.
22. Shao, R.; Li, S.S.; Xin, R.; Geng, S.; Wang, Y.; Oh, S.; Du, S.S.; Lambert, N.; Min, S.; Krishna, R.; et al. Spurious Rewards: Rethinking Training Signals in RLVR. In *Proceedings of the 43rd International Conference on Machine Learning*; PMLR: 2026; Volume 306, pp. 110167–110207.
23. Lyu, Z. Same Answer, New Box: How Much of RLVR's Out-of-Domain Gain Is Answer Format? *arXiv* **2026**, arXiv:2610.20639.
24. Gupta, G.; Tripathi, A. Format Compliance Masquerading as Reasoning Gain in Multimodal RLVR: A Case Study. In Proceedings of the ELLIS Workshop on the Foundations of LLM Post-Training in Changing Environments, NeurIPS 2026. Available online: https://openreview.net/forum?id=aEqsMRb1uQ (accessed on 4 November 2026).
25. Yu, Q.; Tartaglini, A.; Hase, P.; Guestrin, C.; Potts, C. Outcome Rewards Do Not Guarantee Verifiable or Causally Important Reasoning. *arXiv* **2026**, arXiv:2604.22074.
26. Brown, B.; Juravsky, J.; Ehrlich, R.; Clark, R.; Le, Q.V.; Ré, C.; Mirhoseini, A. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. *arXiv* **2024**, arXiv:2407.21787.
27. Yue, Y.; Chen, Z.; Lu, R.; Zhao, A.; Wang, Z.; Yue, Y.; Song, S.; Huang, G. Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? *Adv. Neural Inf. Process. Syst.* **2025**, *38*. https://doi.org/10.52202/085713-1933; arXiv:2504.13837.
28. Zhou, T. When RLVR Shrinks the Reasoning Boundary: Diagnosing Pass@k Inversion. *arXiv* **2026**, arXiv:2607.20543.
29. Huan, M.; Li, Y.; Zheng, T.; Xu, X.; Kim, S.; Du, M.; Poovendran, R.; Neubig, G.; Yue, X. Does Math Reasoning Improve General LLM Capabilities? Understanding Transferability of LLM

Reasoning. In *Proceedings of the 43rd International Conference on Machine Learning*; PMLR: 2026; Volume 306, pp. 46232–46268.

30. Schulman, J.; Moritz, P.; Levine, S.; Jordan, M.I.; Abbeel, P. High-Dimensional Continuous Control Using Generalized Advantage Estimation. In *Proceedings of the International Conference on Learning Representations*; 2016; arXiv:1506.02438.
31. Qwen Team. Qwen2.5: A Party of Foundation Models! Available online: https://qwenlm.github.io/blog/qwen2.5/ (accessed on 12 October 2026).
32. Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; et al. Training Verifiers to Solve Math Word Problems. *arXiv* **2021**, arXiv:2110.14168.
33. Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; Steinhardt, J. Measuring Mathematical Problem Solving with the MATH Dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*; 2021; arXiv:2103.03874.
34. Suzgun, M.; Scales, N.; Schärli, N.; Gehrmann, S.; Tay, Y.; Chung, H.W.; Chowdhery, A.; Le, Q.V.; Chi, E.H.; Zhou, D.; et al. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. In *Findings of the Association for Computational Linguistics: ACL 2023*; Association for Computational Linguistics: Toronto, Canada, 2023; pp. 13003–13051. https://doi.org/10.18653/v1/2023.findings-acl.824.
35. Wang, Y.; Ma, X.; Zhang, G.; Ni, Y.; Chandra, A.; Guo, S.; Ren, W.; Arulraj, A.; He, X.; Jiang, Z.; et al. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. *Adv. Neural Inf. Process. Syst.* **2024**, *37* (Datasets and Benchmarks Track); arXiv:2406.01574.
36. Pan, J. Countdown-Tasks-3to4 Dataset. Available online: https://huggingface.co/datasets/Jiayi-Pan/Countdown-Tasks-3to4 (accessed on 21 February 2025).
37. Sheng, G.; Zhang, C.; Ye, Z.; Wu, X.; Zhang, W.; Zhang, R.; Peng, Y.; Lin, H.; Wu, C. HybridFlow: A Flexible and Efficient RLHF Framework. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*; ACM: Rotterdam, The Netherlands, 2025. https://doi.org/10.1145/3689031.3696075.